Lancaster University Leipzig

Software Design Studio Project II (Networked Studio) LZSCC.311

Morteza Babazadeh[1]

[1]Lancaster University in Leipzig

Week 1 — Michaelmas 2026/27

Adapted from Martin Grimmer

Session Overview

- Module Introduction
- Project: MARP-Guide RAG Chatbot
- Agile Fundamentals: Scrum
- Networked Systems & Development Process
- Team Formation & Feature Selection

Who am I?

- Assistant Professor (Teaching Fellow) at LUL, SCC
- Module Convenor for LZSCC.311
- Office hours: Tuesdays, 10:00–11:00
- Available for questions via email
- And via Moodle

Morteza Babazadeh
m.babazadeh@lancaster.ac.uk

About Networked Studio

- Builds on LZSCC.230 Software Design Studio I (Core Studio)
- Focus on **networked software systems** and **real-world practices**
  - Microservices architecture
  - Event-driven systems
  - Distributed team development
  - Testing
- Equip you with skills for planning, designing and building **mid- to large-scale software systems**
- Train ability to work within **medium-size teams**, coordinate work, and resolve problems
- Train skills in **agile development**, **testing**, **CI**, and **documentation**

Assessment Structure

Assessment 1 (50%)

Upload	11.11.2026, 12:00
Presentations	12.11.2026
Presentation (group)	12.5%
Code & Scrum (group)	12.5%
Reflection (individual)	25%

Focus: Core RAG Pipeline

- Microservices architecture
- Event-driven design ($\geq$3 events)
- Basic RAG ($\geq$1 citation)
- Docker Compose deployment
- Design of the additional features

Assessment 2 (50%)

Upload	16.12.2026, 12:00
Presentations	17.12.2026
Presentation (group)	12.5%
Code & Scrum (group)	12.5%
Reflection (individual)	25%

Focus: Production-Ready System

- Complete MVP ($\geq$2 citations)
- Both features implemented
- Automated tests
- CI pipeline (GitHub Actions)

Assessment Components

Each assessment (50%) has three components:

- **Group presentation** (25%)
- **Group code & Scrum process** (25%), assessed from the submitted repository snapshot
- **Individual reflection** (50%), max. 2 pages + GenAI declaration appendix

Upload to Moodle (each student), the day before the presentations, 12:00:

- Slides and individual reflection (PDF)
- Zip of the code repository **including the `.git` folder** (no build artefacts)

Presentation format:

- Approx. **5 min per team member**, then up to 10 min of questions
- **Architecture** overview and key decisions
- **Live demo**, with the system **already running**
- **Walkthrough** of selected code
- Every member presents and can explain the system and their own work

Standard LU late submission penalties apply to all uploads.

Demonstrating Individual Contribution

Throughout the project, you demonstrate your contribution via:

- Regular and meaningful **Git commits** with clear messages
- **GitHub Issues** created, commented on, and resolved
- **Pull requests** with code reviews and discussions
- **Documentation** contributions visible in repository history
- Participation in **GitHub Projects** and sprint planning activities

Important: Work continuously via GitHub throughout the project period. Git commit timestamps count as submission evidence.

Your **commit history** and the **Scrum artefacts in `/docs`** (from the submitted snapshot) are assessed in the **Code & Scrum Process** component.

GenAI Policy: AMBER Classification

You MAY use Generative AI tools in an ASSISTIVE role only

Permitted (assistive use):

- Chat assistants (e.g., ChatGPT, Claude) for explanations, brainstorming, troubleshooting
- Inline code completion (e.g., GitHub Copilot autocomplete)

Not permitted:

- **Agentic AI tools** that autonomously plan, write, run, or modify code across your repository (e.g., Claude Code, OpenAI Codex, Cursor Agent, Copilot agent mode, Devin)
- **Outsourcing the task:** entering the assessment description into an AI tool to generate the system, documentation, slides, or reflection

GenAI Policy: Requirements

- **Declare usage** in the GenAI declaration appendix of your individual reflection
  - Which tools, for which purposes, how you checked the output
- **Verify correctness** — review and test all AI-assisted code
- **Understand the code** — you must be able to explain any code in your project, regardless of its origin; this is checked in the presentations
- Use of non-permitted tools may be treated as **academic misconduct**

Independent Learning

- Workload per module
  - 30 contact hours (10 weeks $\times$ 3 hours)
  - 270 hours independent learning
  - 300 hours in total (approx. 30 hours per week)
- Most of your studying will be
  - outside of the classroom
  - without your lecturer

- You are expected to work on your project **independently** and **self-driven**
- If you have questions (organisational, technical), ask them
- Start work early and plan ahead — small delays can happen

What we expect from you

- Integrity
  - No plagiarism
  - No faking results
  - Declare GenAI usage
- Effort (active learning)
  - Attend the design studio sessions
  - Ask questions and take notes
  - Read around the subject and try things for yourself
  - Work continuously on your project
  - Plan your time and coursework carefully

What you can expect from us

I'll do my best to:

- Make all notes and materials available on Moodle
- Give you references to follow up
- Offer feedback during Sprint reviews
- Respond to email (ideally as a last resort)

Support Structure

- **Weeks 1–10:** Weekly 3-hour design studio sessions
  - Short input (max. 1 hour), then hands-on studio work in your team
  - Technical and non-technical questions
  - Sprint planning, review and retrospective sessions
- **Office hours:** Tuesdays, 10:00–11:00

- **Week 5:** Upload 11.11.2026, 12:00; Assessment 1 presentations 12.11.2026 (first Sprint Review)
- **Week 6:** Sprint 1 retrospective, Sprint 2 planning
- **Week 10:** Upload 16.12.2026, 12:00; Assessment 2 presentations 17.12.2026 (second Sprint Review)

- If there are additional questions, just contact me

Exceptional Circumstances

- Exceptional Circumstances are **unanticipated events** that significantly impact your ability to complete assessments
- Common situations **not considered exceptional:**
  - Common cold
  - Assessment deadline proximity
  - Normal exam nerves
  - Minor sports injuries
  - Difficulties with housemates/landlords

- **EC Request Procedure:**
  - Submit EC form with **evidence** (e.g., doctor's note)
  - Send to Student Services: `studentservices@lancasterleipzig.de`
  - **Within 3 calendar days from assessment deadline**

Session Overview

- Module Introduction
- Project: MARP-Guide RAG Chatbot
- Agile Fundamentals: Scrum
- Networked Systems & Development Process
- Team Formation & Feature Selection

Product Goal

Build a **chat application** that answers questions about Lancaster University's **Manual of Academic Regulations and Procedures (MARP)**.

Target Audience: Students and staff who need quick access to university regulations.

Value: Reliable, quickly accessible information with sources — no hallucinations.

Key Requirements:

- Answers derived from MARP PDF documents
- Properly cited (title, page number, and link)
- Presented in an understandable manner
- If no suitable source exists → explicitly state "not certain / no source found"

Framework and Constraints

- **Team Size:** 3–4 persons
- **Duration:** 10 weeks (300 hours per person)
- **Methodology:** Scrum (sprints, reviews, retrospectives, visible artefacts)
- **Architecture:** Microservice architecture **and** Event-Driven Architecture (EDA)
  - Decoupled services for ingestion, indexing/retrieval, and chat orchestration
  - All inter-service communication via network protocols (HTTP REST or message queues)
- **RAG (Retrieval-Augmented Generation):** Mandatory
  - Answers must **only** contain information supported by sources
- **LLM:** Use **OpenRouter** (free models permitted, see OpenRouter Documentation on free models rate limits)
- **Data Source:** MARP PDFs from MARP

Minimum Functionality (MVP)

1. **Ingestion:** Automatically discover and fetch MARP PDFs, storing text and metadata (title, URL, date, page numbers)
2. **Indexing:** Implement chunking and embeddings (choice of model is yours), preserve metadata
3. **Retrieval:** Return top-k snippets reproducibly via API
4. **RAG Answers:** Provide chat endpoint/UI that generates answers with $\geq$**2 citations** (title, page, link)
   - *Assessment 1: $\geq$1 citation; Assessment 2: $\geq$2 citations*
5. **EDA Flow:** Key workflow steps must generate events (e.g., `DocumentDiscovered`, `DocumentExtracted`, `ChunksIndexed`, `AnswerGenerated`)
6. **Operations:** Local deployment via **Docker Compose**. Health checks for each service
7. **Quality:** Logs for all services

Technology Constraints

Prohibited Technologies:

- **RAG Frameworks:** OpenWebUI, anything-llm, Haystack, LlamaIndex
- **AI Agent Frameworks:** LangChain, LangGraph, Pydantic AI, CrewAI, ...
- **Pre-built Chat Applications:** templates providing more than 50% of required functionality

Permitted Technologies:

- **HTTP/API Clients:** requests, httpx, fetch (for OpenRouter API calls)
- **UI Frameworks:** React, Vue, Svelte, Angular, Material-UI, Chakra UI
- **Backend Frameworks:** FastAPI, Flask, Express.js, NestJS
- **Infrastructure:** RabbitMQ, Kafka, Redis Pub/Sub, Qdrant, ChromaDB, pgvector
- **ML/NLP Libraries:** sentence-transformers, Hugging Face transformers, PyPDF2, pdfplumber

Core Implementation Requirements

The following components **must be implemented from scratch** by your team:

1. RAG Pipeline Core:

- Custom chunking strategy with justification
- Custom retrieval orchestration logic (query $\rightarrow$ embedding $\rightarrow$ search $\rightarrow$ ranking)
- Custom citation extraction and formatting logic
- Custom prompt templates for RAG responses

2. Event-Driven Architecture:

- $\geq$5 custom event types defined with schemas
- Custom event handler implementations
- Documented event flow through the system

3. Microservices Architecture:

- $\geq$3 independent services with clear responsibilities
- Custom-designed REST/GraphQL APIs with documentation
- Inter-service communication (both synchronous and event-based)

Required Additional Features

Each team must implement **exactly 2 additional features** beyond the MVP:

- **1× Feature from Tier 1 (Foundation Features)** — extends basic infrastructure
- **1× Feature from Tier 2 (Advanced Features)** — adds significant technical complexity

Timeline:

- **Today (Week 1):** Select your features
- **Assessment 1 (12.11.2026):** Document the design in the `/docs` folder
- **Assessment 2 (17.12.2026):** Demonstrate the working implementation

Tier 1: Foundation Features

Choose exactly ONE:

1. **User Authentication & Session Management:** Login/logout, session handling, user-specific chat isolation
2. **Chat History Persistence:** Store all chats in database, retrieve past conversations
3. **Bookmarks & Favourites:** Users can mark important messages, retrieve saved items
4. **Document Management Interface:** Admin interface for manual PDF upload, document status overview
5. **Basic Monitoring Dashboard:** Service health checks, indexing statistics, request counts

These features extend the basic infrastructure and provide foundation for production readiness.

Tier 2: Advanced Features

Choose exactly ONE:

1. **Hybrid Search:** Combine dense vector search with sparse retrieval (BM25), weighted fusion
2. **Query Rewriting & Expansion:** Automatic query reformulation, synonym expansion, LLM-based rewriting
3. **Re-Ranking Pipeline:** Two-stage retrieval (recall $\rightarrow$ precision) with separate re-ranking model
4. **Multi-Model Comparison:** Parallel answer generation with different LLMs, side-by-side UI comparison
5. **User Feedback & Quality Metrics:** Thumbs up/down, feedback tracking, confidence scoring
6. **Advanced Monitoring & Observability:** Distributed tracing, latency metrics per service, event flow visualisation
7. **Rate Limiting & Intelligent Caching:** Request limits per user/service,

Acceptance Criteria (Definition of Done)

- **Correctness & Sources:** $\geq$90% of answers in the demo catalogue contain **correct** citations (title + page + link)
- **Fallback:** If no suitable source exists $\rightarrow$ explicitly state "not certain / no source found"
- **Reproducibility:** `docker compose up` brings the system up and running (README with commands)
- **Network Architecture:** All services communicate via network protocols (HTTP REST APIs or message queues); no in-process communication
- **Technology Compliance:** No prohibited frameworks used; core RAG pipeline, EDA, and microservices implemented from scratch
- **Additional Features:** Both required features (1$\times$ Tier 1, 1$\times$ Tier 2) documented in Assessment 1, fully implemented in Assessment 2
- **Scrum Evidence:** Product goal, prioritised backlog, sprint artefacts, reviews/retrospectives documented

Example Milestones: Sprint 1 (Weeks 1–5)

- **M1 (Week 1):** Team formation; Scrum fundamentals; Sprint planning; feature selection (Tier 1 + Tier 2); repository setup + GitHub Projects configured
- **M2 (Week 2):** Microservices architecture designed; tech decisions documented; ingestion service started
- **M3 (Week 3):** Ingestion service functional (discover + fetch MARP PDFs); Docker Compose setup; EDA applied; first events (`DocumentDiscovered`, `DocumentExtracted`)
- **M4 (Week 4):** Indexing service (chunking + embeddings); retrieval service (`/search` endpoint); basic RAG (`/chat` endpoint with 1 citation); 3+ events total
- **M5 (Week 5):** System integration verified; Docker Compose fully functional; **Assessment 1 presentation (12.11.2026)**

Example Milestones: Sprint 2 (Weeks 6–10)

- **M6 (Week 6):** Sprint 1 retrospective completed; Sprint 2 planning finished; additional features work started
- **M7 (Week 7):** Chat UI implemented; complete RAG (2 citations); 5+ events implemented; automated tests started; CI pipeline planned
- **M8 (Week 8):** Additional features (Tier 1 + Tier 2) functional; automated tests implemented; GitHub Actions CI pipeline operational
- **M9 (Week 9):** Test catalogue (10–15 questions) completed; acceptance criteria verified; final integration and bug fixes
- **M10 (Week 10):** Final system polish and documentation; **Assessment 2 presentation (17.12.2026)**

Session Overview

- Module Introduction
- Project: MARP-Guide RAG Chatbot
- Agile Fundamentals: Scrum
- Networked Systems & Development Process
- Team Formation & Feature Selection

What is Scrum?

Scrum is an agile framework for managing complex work

Core Principles:

- Empirical process control (transparency, inspection, adaptation)
- Iterative and incremental development
- Self-organising teams
- Regular feedback and continuous improvement

For this module:

- 2 sprints $\times$ 5 weeks each
- Sprint 1: Weeks 1–5 (Assessment 1 on 12.11.2026)
- Sprint 2: Weeks 6–10 (Assessment 2 on 17.12.2026)

Scrum Roles

Product Owner:

- Defines product vision and goals
- Manages product backlog
- *In this module: You collectively, guided by acceptance criteria*

Development Team:

- Self-organising, cross-functional team
- Delivers working increments
- *In this module: Your team of 3–4 students*

Scrum Master:

- Facilitates Scrum process
- Removes impediments
- *In this module: You collectively + module convenor as facilitator*

Scrum Ceremonies (Events)

Sprint Planning:

- Define sprint goal and select backlog items
- *In this module: Week 1 (today) and Week 6*

Daily Standup:

- Brief daily synchronisation (What did I do? What will I do? Any blockers?)
- *In this module: Self-organised within your team*

Sprint Review:

- Demonstrate working increment
- *In this module: Assessment 1 and Assessment 2 presentations*

Sprint Retrospective:

- Reflect on process and plan improvements
- *In this module: After Assessment 1 (Week 6)*

Scrum Artefacts

Product Backlog:

- Ordered list of everything needed in the product
- Continuously refined and prioritised
- *In this module: Use GitHub Issues*

Sprint Backlog:

- Set of backlog items selected for this sprint
- Plan for delivering the increment
- *In this module: Use GitHub Projects (Sprint board)*

Increment:

- Sum of all backlog items completed during a sprint
- Must meet Definition of Done (acceptance criteria)
- *In this module: Working system demonstrated at assessments*

GitHub Issues for Project Management

GitHub Issues are used to track work items:

- Create issues for user stories, tasks, bugs, and technical debt
- Use labels to categorise (e.g., `epic`, `feature`, `bug`, `documentation`)
- Assign issues to team members
- Reference issues in commits (`Fixes #42`)

Example user story format:

Title: *Implement document ingestion service*

Description: *As a system, I want to automatically discover and fetch MARP PDFs so that they can be indexed for retrieval.*

Acceptance criteria:

- *PDFs are discovered from MARP main page*
- *PDFs are downloaded and stored with metadata*
- *`DocumentDiscovered` event is emitted*

GitHub Projects for Sprint Planning

GitHub Projects provide a Kanban style board for sprint management:

- Create a project for each sprint
- Columns: Backlog, To Do, In Progress, In Review, Done
- Move issues across columns as work progresses
- Visualise sprint progress and bottlenecks

Sprint workflow:

1. Sprint planning: Move selected issues from Backlog to To Do
2. Daily work: Move issues to In Progress when starting work
3. Code review: Move to In Review when PR is created
4. Completion: Move to Done when PR is merged and acceptance criteria met

Documenting Scrum Artefacts

For assessment, you must document Scrum artefacts in `/docs` folder:

Required documentation:

- **Product goal** and vision
- **Product backlog** (export of your GitHub Issues — a link alone is not enough)
- **Sprint log** for each sprint:
  - Sprint goal
  - Sprint backlog (selected issues)
  - Sprint review summary (what was completed)
- **Retrospective notes** (after Sprint 1):
  - What went well?
  - What didn't go well?
  - What will we improve in Sprint 2?

Together with your commit history, these artefacts are assessed in the **Code &**

Session Overview

- Module Introduction
- Project: MARP-Guide RAG Chatbot
- Agile Fundamentals: Scrum
- Networked Systems & Development Process
- Team Formation & Feature Selection

What is a Networked System?

A **networked system** consists of multiple software components that communicate over network protocols.

Key Characteristics:

- **Distributed:** Components run on different processes or machines
- **Decoupled:** Services have independent lifecycles
- **Network Communication:** HTTP REST, message queues, gRPC, etc.
- **Resilient:** Fault tolerance and graceful degradation

For this module:

- All services communicate via network protocols
- No in-process communication (e.g., function calls between services)
- Explicit network boundaries and API contracts

Microservices Architecture

Microservices: Architectural style structuring an application as a collection of loosely coupled services

Benefits:

- Independent deployment and scaling
- Technology diversity (different languages/frameworks per service)
- Fault isolation
- Team autonomy

Challenges:

- Distributed system complexity
- Inter-service communication overhead
- Data consistency across services
- Testing and debugging

Event-Driven Architecture (EDA)

Event-Driven Architecture: Services communicate via events (state changes or significant occurrences)

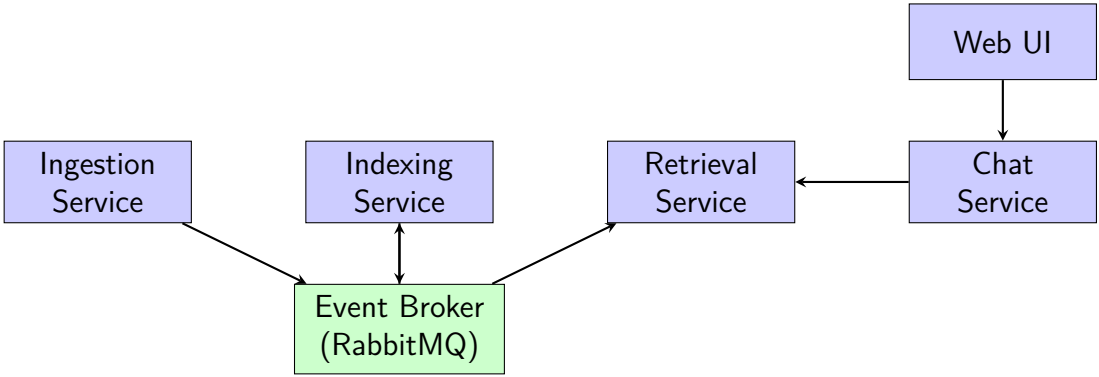
Key Concepts:

- **Event:** Immutable fact about something that happened (past tense); a request that needs an immediate answer is an **API call** instead
- **Event Producer:** Service that publishes events
- **Event Consumer:** Service that subscribes to and handles events
- **Event Broker:** Message queue or pub/sub system (RabbitMQ, Kafka, Redis)

Example for MARP-Guide:

- Ingestion service publishes `DocumentDiscovered` event
- Extraction service consumes event, processes PDF, publishes `DocumentExtracted`

Example Architecture: MARP-Guide RAG



Note: This is a simplified example. Your architecture will be more detailed.

Continuous Integration & Testing

Continuous Integration (CI): Integrate code frequently, test automatically

Benefits:

- Early detection of integration issues
- Automated quality checks
- Faster feedback loops
- Reduced integration pain

For this module:

- Use **GitHub Actions** for CI
- Merge to `develop` branch at least daily
- Automated tests run on every push
- Merge to `main` only for releases (Assessment 1, Assessment 2)
- Tag the submitted commit as `assessment-1` / `assessment-2`; the zipped snapshot (incl. `.git`) is what is marked

Branching Strategy

Recommended branching model:

- `main` **branch:** Production-ready releases
  - Merge only for Assessment 1 and Assessment 2
  - Always deployable and passing all tests
- `develop` **branch:** Integration branch for ongoing work
  - Merge feature branches here at least daily
  - Should always be in working state
- **Feature branches:** Individual work on features/tasks
  - Named `feature/issue-number-description`
  - Create pull request to merge into `develop`
  - Code review before merge

Docker and Containerisation

Docker: Platform for developing, shipping, and running applications in containers

Benefits:

- Environment isolation (no dependency conflicts)
- Fast deployment due to automation
- Scalability
- Consistent environments (dev/test/prod parity)

For this module:

- Each service runs in its own Docker container
- `docker-compose.yml` defines all services
- `docker compose up` brings entire system online
- Include health checks for each service

Executable Source Code

Your repository must be immediately executable:

- **README.md** with clear setup and run instructions
- **docker-compose.yml** defining all services
- **Dockerfiles** for each service
- **Build tool wrappers** (if applicable):
  - `pom.xml` and `mvnw` (Maven)
  - `build.gradle` and `gradlew` (Gradle)
  - `package.json` (npm/yarn)
  - `requirements.txt` or `pyproject.toml` (Python)

Important: Never commit build artefacts (`node_modules/`, `.venv/`, `target/`, etc.)

Professional Development Practices

- **Version Control:** Regular, meaningful commits with clear messages
- **Code Review:** Pull requests reviewed by at least one team member
- **Testing:** Write tests before or alongside implementation (TDD)
- **Documentation:** Inline comments, API specs, architecture diagrams
- **Logging:** Structured logs for debugging and monitoring
- **Error Handling:** Graceful degradation and informative error messages

Remember: In 3rd year, we expect you to develop code professionally.

Session Overview

- Module Introduction
- Project: MARP-Guide RAG Chatbot
- Agile Fundamentals: Scrum
- Networked Systems & Development Process
- Team Formation & Feature Selection

Team Assignments

Teams will be announced shortly.

Once teams are assigned:

- Introduce yourselves
- Exchange contact information
- Set up communication channels (Discord, Slack, etc.)
- Agree on regular meeting times

First team task today:

1. Create GitHub repository
2. Set up GitHub Projects
3. Select Tier 1 and Tier 2 features
4. Create initial backlog (GitHub Issues)

Activity: Feature Selection

Your task (30 minutes):

1. As a team, review Tier 1 and Tier 2 feature descriptions
2. Discuss which features align with your team's skills and interests
3. Consider technical complexity and time constraints
4. Select:
   - **1× Tier 1 feature** (Foundation)
   - **1× Tier 2 feature** (Advanced)
5. Document your choices in a GitHub Issue

Considerations:

- Which features provide most learning value?
- Which features best demonstrate your skills?
- Which features are feasible within 10 weeks?

Next Steps: Sprint 1 Planning

Before Week 2:

- Set up GitHub repository with README
- Create GitHub Projects for Sprint 1
- Create initial backlog (GitHub Issues) based on suggested EPICs:
  - EPIC Ingestion
  - EPIC Extraction
  - EPIC Index
  - EPIC Retrieval
  - EPIC RAG & Chat
  - EPIC UX
  - EPIC Quality
- Begin architectural design discussions

Week 2 focus: Requirements and microservices architecture

Reading Material

Recommended readings:

1. **The Scrum Guide** by Ken Schwaber and Jeff Sutherland
   `https://www.scrum.org/resources/scrum-guide`
2. **Microservices Patterns** by Chris Richardson
3. **Practical Event-Driven Microservices Architecture** by Hugo Filipe Oliveira Rocha
4. **Agile Software Development, Principles, Patterns, and Practices** by Robert C. Martin
5. **Continuous Integration (CI) and Continuous Delivery (CD): A Practical Guide** by Henry van Merode

You are **not required to purchase** these books, but they provide valuable background.

Summary: Week 1

What we covered today:

- Module structure and assessment (2 increments, 50% each; presentations on 12.11. and 17.12.2026)
- Project goal: MARP-Guide RAG Chatbot
- Technology constraints and MVP requirements
- Scrum fundamentals and GitHub project management
- Networked systems, microservices, and EDA
- Development practices (CI/CD, Docker)

Your tasks for next week:

- Set up GitHub repository and projects
- Select Tier 1 and Tier 2 features
- Create initial product backlog
- Begin architecture design discussions